

Governance (Access, Permissions and Security)

Original slides belong to: Amit Dandekar And Dr. Hossein Saiedian

Contents

- Security requirements
- SQL security model
- Data sensitivity
- Security vs. Precision
- Inference & aggregation problem
- Multilevel databases
- Future direction

Database Concepts

- **Structured Query Language (SQL)**
 - to manipulate relations and data in a relational database
- **Types of SQL Commands**
 - **Data Dictionary Language (DDL)**
 - define, maintain, drop schema objects
 - **Data Manipulation Language (DML)**
 - SELECT, INSERT, UPDATE
 - **Data Control Language (DCL):**
 - **Control security (GRANT,REVOKE) and concurrent access (COMMIT , ROLLBACK)**

Security Requirements

- Physical database integrity
- Logical database integrity
- Element integrity
- Auditability
- Access control
- User authentication
- Availability

Security Requirements

- Physical database integrity
 - immunity to physical catastrophe, such as power failures, media failure
 - physical securing hardware, UPS
 - regular backups
- Logical database integrity
 - reconstruction Ability
 - maintain a log of transactions
 - replay log to restore the systems to a stable point

Security Requirements

- Element integrity

- integrity of specific database elements is their correctness or accuracy
 - field checks
 - allow only acceptable values
 - access controls
 - allow only authorized users to update elements
 - change log
 - used to undo changes made in error
 - referential Integrity (key integrity concerns)
 - two phase locking process

- Auditability

- log read/write to database

Security Requirements

- Access Control (similar to OS)
 - logical separation by user access privileges
 - more complicated than OS due to complexity of DB
- User Authentication
 - may be separate from OS
 - can be rigorous
- Availability
 - concurrent users
 - granularity of locking
 - reliability

SQL Security Model

- SQL security model implements **DAC (Discretionary Access Control)** based on
 - *users*: users of database - *user identity* checked during login process;
 - *actions*: including **SELECT**, **UPDATE**, **DELETE** and **INSERT**;
 - *objects*: tables (*base relations*), views, and columns (*attributes*) of tables and views
- Users can protect objects they own
 - when object created, a user is designated as ‘owner’ of object
 - owner may grant access to others
 - users other than owner have to be granted *privileges* to access object

DAC

Discretionary Access Control (DAC)



What are the basic principles of DAC?

- Object characteristics (size, name, directory path) are invisible to users that aren't authorized.
- Several failed access attempts enforce additional multi-factor authentication or deny access.
- Users can transfer their object ownership to other users. Also, the owner determines the access type of other users. Based on these access privileges, the operating system decides whether to grant access to a file.

Gaining access to a file in the DAC model works like this:

- User 1 creates a file and becomes its owner or obtains access rights to an existing file.
- User 2 requests access to this file. They provide their credentials: username, password, or something else.
- User 1 grants access at their discretion. However, user 1 can't grant access rights that exceed their own. For example, if user 1 can only read a document, they can't allow user 2 to edit it.
- If there's no contradiction between the access control created by an administrator and the decision made by user 1, access is granted.

SQL Security Model

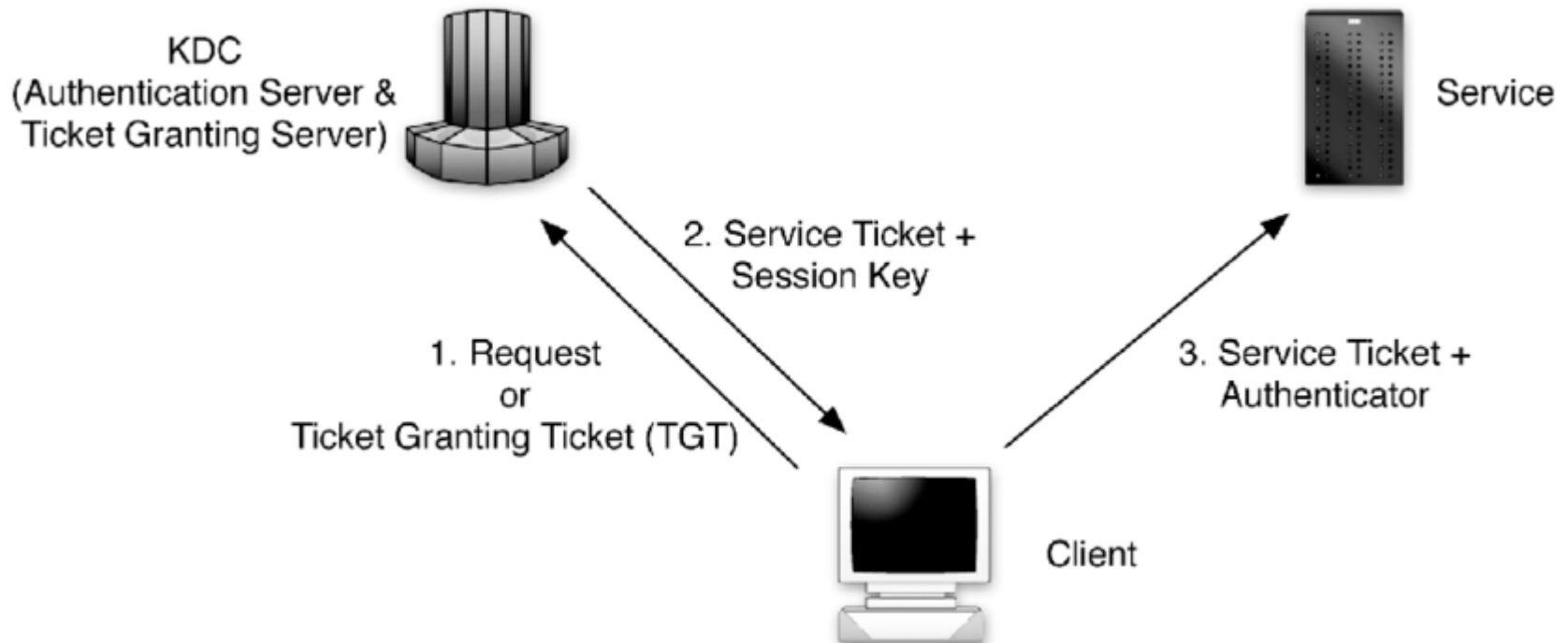
- Components of privilege are
 - grantor, grantee, object, action, grantable
 - privileges managed using **GRANT** and **REVOKE** operations
 - the right to grant privileges can be granted
- Issues with privilege management
 - each grant of privileges is to an individual or to “Public”
 - makes security administration in large organizations difficult
 - individual with multiple roles may have too many privileges for one of the roles

SQL Security Model

- Authentication & identification mechanisms
 - CONNECT <user> USING<password>
 - DBMS may chose OS authentication
 - or its own authentication mechanism
 - Kerberose
 - is a computer-network authentication protocol that works on the basis of tickets to allow nodes communicating over a non-secure network to prove their identity to one another in a secure manner.
 - PAM
 - privileged access management

SQL Security Model

- Kerberos
 - is a computer-network authentication protocol that works on the basis of tickets to allow nodes communicating over a non-secure network to prove their identity to one another in a secure manner.



SQL Security Model

- Access control through views

- many security policies better expressed by granting privileges to views derived from base relations

- example

```
CREATE VIEW AVSAL(DEPT, AVG)  
AS SELECT DEPT, AVG(SALARY)  
FROM EMP GROUP BY DEPT
```

- access can be granted to this view for every dept mgr

- example

```
CREATE VIEW MYACCOUNT AS  
SELECT * FROM Account  
WHERE Customer = current_user()
```

- view containing account info for current user

SQL Security Model

- Advantages of views
 - views are flexible, and allow access control to be defined at a description level appropriate to application
 - views can enforce context and data-dependent policies
 - data can easily be reclassified

SQL Security Model

- Disadvantages of views
 - access checking may become complex
 - views need to be checked for correctness (do they properly **capture policy**?)
 - completeness and consistency not achieved automatically - **views may overlap or miss parts of database**
 - security-relevant part of DBMS may become very large (view results are stored)

SQL Security Model

- Inherent weakness of **DAC (discretionary access control)**
 - DAC is very flexible and suitable model for DBMS.
 - As the name implies, DAC allows the creators or owners of files to assign access rights.
 - Also, a user (or subject) with discretionary access to information can pass that information on to another user (or subject)
 - Trojan horses to copy information from one object to another

SQL Security Model

- Mandatory Access Controls (MAC)

- Mandatory Access Control (MAC) is system-enforced access control based on a subject's clearance and an object's labels. Subjects and Objects have clearances and labels, respectively, such as confidential, secret, and top secret. **A subject may access an object only if the subject's clearance is equal to or greater than the object's label.** Subjects cannot share objects with other subjects who lack the proper clearance, or “write down” objects to a lower classification level (such as from top secret to secret). MAC systems are usually focused on preserving the confidentiality of data.

- No read up, no write down
- Expensive to implement
- Traditional MAC implementations in RDBMS have focused solely on MLS (Multi-level Security)
- There have been three commercial MLS RDBMS offerings
 - Trusted Oracle, Informix OnLine/Secure, Sybase Secure SQL Server

Mandatory Access Controls (MAC)

Mandatory Access Control (MAC)



Mandatory Access Controls (MAC)

What are the basic principles of MAC?

- The utmost privacy and confidentiality of the organization's resources are paramount.
- No one has default privileges to access or edit someone's data.
- Access provisioning is centrally administered.
- Each individual and resource in the system has security labels with their classification and category.

With MAC, the process of gaining access looks like this:

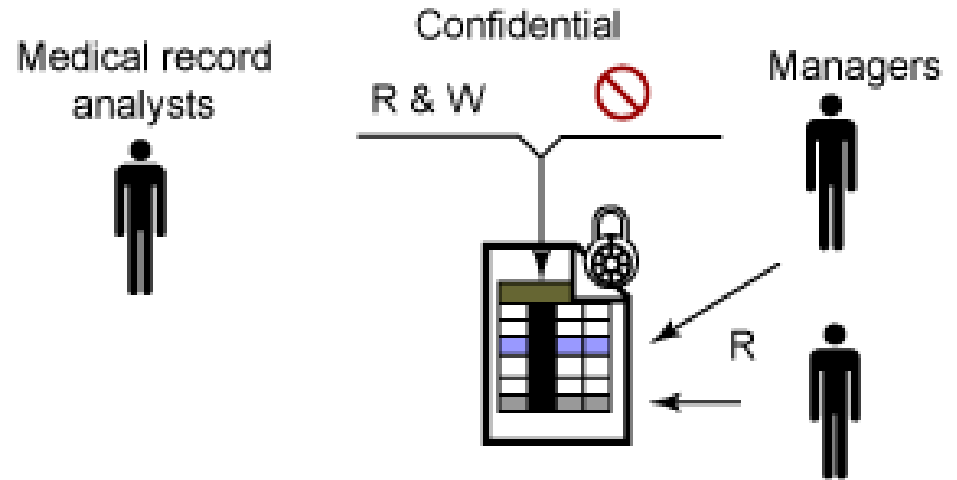
- The administrator configures access policies and defines security attributes: confidentiality levels, clearances for accessing different projects and types of resources.
- The administrator assigns each subject (user or resource that accesses data) and object (file, database, port, etc.) a set of attributes.
- When a subject attempts to access an object, the operating system examines the subject's security attributes and decides whether access can be granted.
- To obtain access to the object, the user provides their credentials.

Case Study

RECORDID	CLIENTNO	DEPTNO	ALLOCATION_DATE	LAST_UPDATE	MEDICAL_HISTORY	RISK_FACTOR
0010	K108341	K01	2006/01/05	2006/02/05	Diabetes	0
0020	K104546	K01	2006/10/20	2006/11/05	Arthritis	2
0030	S245987	S02	2006/09/01	2006/10/05	High Blood Pressure	3
0040	S245456	S02	2006/06/26	2006/07/05	Asthma	1

- Medical record analyst
 - READ all records
 - WRITE all records
- Managers
 - READ client records of their department
 - READ only non-confidential columns
 - No WRITE access

Case Study



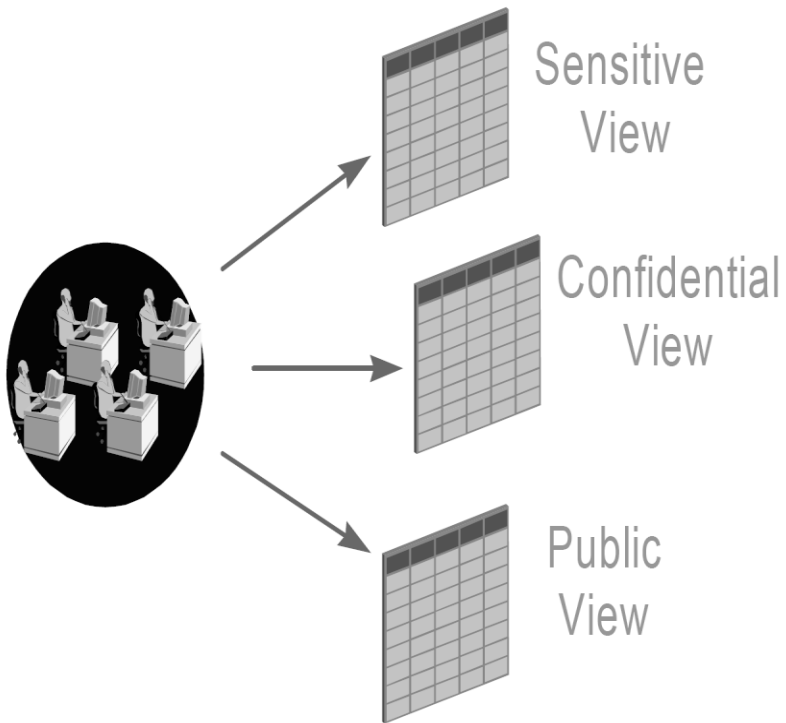
- Columns
 - medical record analysts have READ/WRITE access to confidential columns
 - managers have READ access to non-confidential columns
- Rows:
 - medical record analysts can read and update all the records
 - managers can read but not update client records for their department

Case Study: DAC Solution

Three approaches used to provide row level security using DAC (Discretionary Access Control)

- Application views
- Programming logic embedded in the application
- Physical separation using one or more databases

Case Study: DAC Solution



- Application views
 - Widely used approach
 - Views provide the ability to filter data.

Case Study: DAC Solution

```
Create view manager_K01 as
select recordid, clientno,
       deptno, allocation_date,
       last_update, risk_factor
from Med_records
where Dept = 'K01';
```

```
Create view manager_S01 as
select recordid, clientno, deptno,
       allocation_date, last_update, risk_factor
from Med_records
where Dept = 'S01';
```

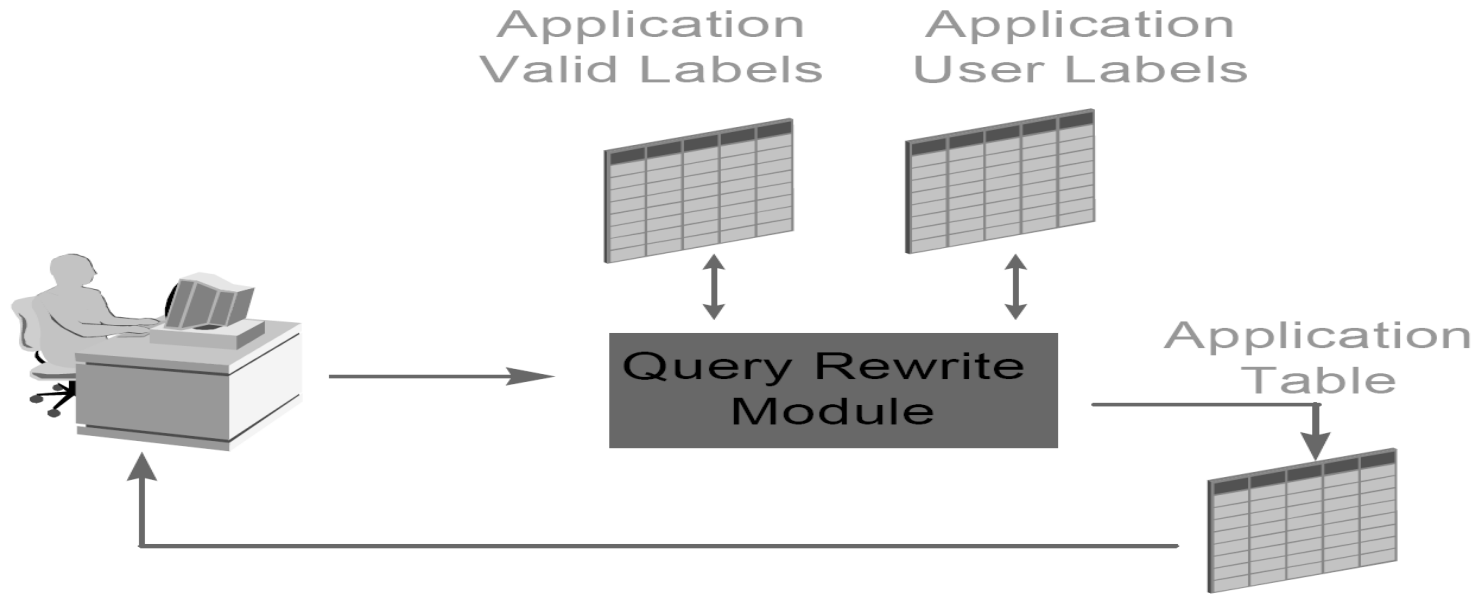
```
Create view med_rec_analyst as
select * from Med_records;
```

Case Study: DAC Solution

- Application views
 - number of views required is sometimes large as application ages
 - directing application users to the correct view becomes management burden
 - application complexity tends to increase due to unforeseen security requirements

Case Study: DAC Solution

Queries can be modified/controlled before execution depending upon the users/authority



- **Application Programmatic Logic Approach**

- in this approach, application controls SQL statements outside the application.

Case Study: DAC Solution

- Application program logic approach
 - SQL statements issued outside the application using utility such as SQL Plus can't be controlled
 - In scenario of application rewriting SQL statements to restrict access based on data sensitivity, typically numerous additional tables must be build
 - Those tables need to be maintained to manage information related authorizations of application user

Case Study: DAC Solution



- Multiple database approach
 - No of databases required is equal to the number of data sensitivities.
 - data can be protected by using dedicated databases to manage each sensitivity

Case Study: DAC Solution

- Multiple database approach
 - number of databases required is equal to the number of data sensitivities
 - overhead created by running multiple databases in terms of memory, processing power and physical storage is substantially increased
 - cost associated of managing single database is multiplied by number of databases
 - viewing information across multiple database requires distributed queries and application logic

Case Study: MAC Solution

- Designing security solution
 - Row and column security labels that protect the columns and rows
 - User security labels that grant users the appropriate access

Case Study: MAC Solution

- revisit the problem

Position	READ	WRITE
Medical record analyst	ALL	ALL
Managers	Client records for their department and only non-confidential columns	None

- To restrict access to the column that is confidential, apply confidential security label to the column
- To restrict managers' access to only the records for their department, each row can be tagged with a security label that indicates the department.
- Write restriction for managers can be implemented by revoking their write privileges.

Case Study: MAC Solution

- A column security label.
- Four security labels for row protection
- User security label for medical record analysts
- Grant security labels to users

SQL Security Model

- Issues with MAC

- Information tends to become over classified
- No protection against violations that produce illegal information flow through indirect means
 - **Inference channels** - A user at a low security class uses the low data to infer information about high security class
 - **Covert channels** - require two active agents, one at a low level and the other at a high level and an encoding scheme

Data Sensitivity

- Sensitive data is data that should *not* be made public
- Factors determining sensitivity
 - Inherently sensitive: the value itself may be so revealing that it is sensitive
 - Locations of defensive missiles
 - From a sensitive source
 - CIA informer whose identity may be compromised
 - Part of a sensitive *attribute* or a sensitive *record*
 - Salary attribute from an HR database
 - Sensitive *with respect to previously disclosed* data
 - Longitude of secret army base if latitude is known

Data Sensitivity

- Even metadata (data about data) may be sensitive
 - Bounds: indicating that a sensitive value, y , is between two values, L and H .
 - Negative result: disclosing that z is *not* the value of y *may be sensitive. Especially when z has only small set of possible values*
 - Existence: existence of data is itself may be sensitive piece of data
 - Probable value: probability that a certain element has a certain value

Security vs. Precision

- Precision: revealing as much non sensitive data as possible
 - disclose as much data as possible
 - Issue: User may put together pieces of disclosed data and infer other, more deeply hidden, data
- Security: reveal only those data that are not sensitive
 - rejecting any query that mentions a sensitive field
 - Issue: may reject many reasonable and non disclosing queries
- The ideal combination : perfect confidentiality with maximum precision
 - achieving this goal is not easy !

Statistical Databases

- A statistical database is a database used for statistical analysis purposes. It is an OLAP (online analytical processing), instead of OLTP (online transaction processing) system.
- Example: medical record database where researchers access only statistical measures
- In a statistical database, information retrieved by means of statistical (aggregate) queries on an attribute

Statistical Databases

- In a statistical database, it is often desired to allow query access only to aggregate data, not individual records.
- Securing such a database is a difficult problem, since intelligent users can use a combination of aggregate queries to derive information about a single individual.

Inference

- Security issue with statistical databases
 - Inference problem exists when sensitive data can be deduced from non sensitive data
 - Attacker combines information from outside the database with database responses

Inference

Anonymous medical data:

SSN	Name	Race	DOB	Sex	Zip	Marital	Heath
		Asian	09/07/64	F	22030	Married	Obesity
		Black	05/14/61	M	22030	Married	Obesity
		White	05/08/61	M	22030	Married	Chest pain
		White	09/15/61	F	22031	Widow	Aids

Public available voter list:

Name	Address	City	Zip	DOB	Sex	Party
....						
Sue Carlson	900 Market St.	Fairfax	22031	09/15/61	F	Democrat

Sue Carlson has Aids!

Inference

- Types of attack

- Direct attack: aggregate computed over a small sample so individual data items leaked
- Indirect attack: infer sensitive info from statistics (sum, count, median) also from info external to the attacked DB
 - Tracker attack: (intersection of sets)
 - Linear system vulnerability: takes tracker attacks further, using algebraic relations between query sets to construct equations yielding desired information

Inference

NAME	SEX	RACE	AID	FINES	DRUGS	DORM
-------------	------------	-------------	------------	--------------	--------------	-------------

Adams	M	C	5000	45	1	Holmes
Bailey	M	B	0	0	0	Grey
Chin	F	A	3000	20	0	West
Dewitt	M	B	1000	35	3	Grey
Earhart	F	C	2000	95	1	Holmes
Fein	F	C	1000	15	0	West
Groff	M	C	4000	0	3	West
Hill	F	B	5000	10	2	Holmes
Koch	F	C	0	0	1	West
Liu	F	A	0	10	2	Grey
Majors	M	C	2000	0	2	Grey

Inference

- Direct Attack

- Determine values of sensitive fields by seeking them directly with queries that yield few records

LIST NAME where SEX=M and DRUGS=1

- “n items over k percent” rule helps prevent attack

Inference

Indirect attack: combines several aggregates

Sums of Financial Aid by Dorm and Sex				
	Holmes	Grey	West	Total
M	5000	3000	4000	12000
F	7000	0	4000	11000
Total	12000	3000	8000	23000

Students by Dorm and Sex				
	Holmes	Grey	West	Total
M	1	3	1	5
F	2	1	3	6
Total	3	4	4	11

- 1 Male in Holmes receives 5000
- 1 Female in Grey received no aid

Inference

- Often databases protected against delivering small response sets to queries
- Trackers can identify unique value
 - request (n) and $(n-1)$ values
 - given n and $n - 1$, we can easily compute the desired single element

Inference

- How many Caucasian females live in Holmes Hall?
 - $\text{Count}((\text{sex}=\text{f}) \wedge (\text{race}=\text{c}) \wedge (\text{dorm}=\text{holmes}))$
 - Result: refused because one record dominates the result
 - Now issue two queries on database
 - $\text{Count}(\text{sex}=\text{f})$ response = 6
 - $\text{Count}((\text{sex}=\text{f}) \wedge (\text{RACE} \neq \text{C}) \wedge (\text{DORM} \neq \text{holmes}))$ response=5
 - Thus $5-4=1$ females live in holmes hall

Inference

- Tracker is a specific case of 'Linear system vulnerability'
 - result of the query is a set of records
 - $q1 = c1 + c2 + c3 + c4 + c5$
 - $q2 = c1 + c2 + c4$
 - $q3 = c3 + c4$
 - $q4 = c4 + c5$
 - $q5 = c2 + c5$
 - we can obtain $c5 = ((q1 - q2) - (q3 - q4))/2$
 - all other c can be derived

Inference

- Protection techniques
- Only queries disclosing non sensitive data allowed
- Controls applied to individual items within the database

Multilevel Databases

- Data sensitivity not black or white
 - Exist shades of sensitivity
 - **Grades of security may be needed**
- So far we have seen sensitivity a function of the *attribute* (column)
 - E.G. 'Drug use' column sensitive
- Actually sensitivity *not* function of column or row
 - The security of one element may be different from that of other elements of the same row or column
 - **Security implemented for each individual element**

Multilevel Databases

Data and Attribute Sensitivity				
Name	Department	Salary	Phone	Performance
Rogers	training	43,800	4-5067	A2
Jenkins	research	62,900	6-4281	D4
Poling	training	38,200	4-4501	B1
Garland	user services	54,600	6-6600	A4
Hiltten	user services	44,500	4-5351	B1
Davis	admin	51,400	4-9505	A3

Multilevel Databases

- Leads to Multi Level Security Model
 - *n levels* of sensitivity
 - Objects separated into compartments by category
 - Sensitivity marked for each value in database
 - Every *combination of elements* can also have a distinct sensitivity
 - Access control policy dictate which users may have access to what data

Multilevel Databases

- To preserve Integrity, DBMS must enforce “No write down” (property)
 - the process that reads high level data cannot write to a lower level

Multilevel Databases

- Polyinstantiation
 - different users operating at two different levels of security might get two different answers to the same query
 - one record can appear (be instantiated) many times, with a different level of confidentiality each time

Polyinstantiated Records			
Name	Sensitivity	Assignment	Location
Hill, Bob	C	Program Mgr	London
Hill, Bob	TS	Secret Agent	South Bend

Case Study

RECORDID	CLIENTNO	DEPTNO	ALLOCATION_DATE	LAST_UPDATE	MEDICAL_HISTORY	RISK_FACTOR
0010	K108341	K01	2006/01/05	2006/02/05	Diabetes	0
0020	K104546	K01	2006/10/20	2006/11/05	Arthritis	2
0030	S245987	S02	2006/09/01	2006/10/05	High Blood Pressure	3
0040	S245456	S02	2006/06/26	2006/07/05	Asthma	1

- Medical record analyst
 - READ all records
 - WRITE all records
- Managers
 - READ client records of their department
 - READ only non-confidential columns
 - No WRITE access

Case Study: MAC Solution

Example of steps to implement (Label-based Access Control) LBAC:

1. Defining the security policies and labels

a. Defining the security label component

```
CREATE SECURITY LABEL COMPONENT SLC_LEVEL
```

```
SET {'CONFIDENTIAL'}
```

```
CREATE SECURITY LABEL COMPONENT SLC_LIFEINS_ORG TREE  
{'LIFE_INS_DEPT' ROOT,
```

```
'K01' UNDER 'LIFE_INS_DEPT',
```

```
'K02' UNDER 'LIFE_INS_DEPT',
```

```
'S01' UNDER 'LIFE_INS_DEPT',
```

```
'S02' UNDER 'LIFE_INS_DEPT' }
```

b. Defining the security policy

```
CREATE SECURITY POLICY MEDICAL_RECORD_POLICY COMPONENTS  
SLC_LEVEL, SLC_LIFEINS_ORG
```

```
WITH DB2LBACRULES
```

```
RESTRICT NOT AUTHORIZED WRITE SECURITY LABEL
```


Case Study: MAC Solution

C. Defining the security labels

```
CREATE SECURITY LABEL  
MEDICAL_RECORD_POLICY.MED_RECORD COMPONENT  
SLC_LEVEL 'CONFIDENTIAL'
```

For each department,

```
CREATE SECURITY LABEL  
MEDICAL_RECORD_POLICY.LIFEINS_DEPT_K01  
COMPONENT SLC_LIFEINS_ORG 'K01'
```

For Medical analyst

```
CREATE SECURITY LABEL  
MEDICAL_RECORD_POLICY.MEDICAL_ANALYST  
COMPONENT SLC_LEVEL 'CONFIDENTIAL',  
COMPONENT SLC_LIFEINS_ORG 'K01', 'K02', 'S01',  
'S02'
```

Case Study: MAC Solution

2. Altering the MEDICAL_RECORD table by adding a security label column for row level protection, marking the confidential column as protected, and attaching the security policy to the table.

```
GRANT SECURITY LABEL  
MEDICAL_RECORD_POLICY.MEDICAL_ANALYST TO USER  
<administrator_auth_id> FOR ALL ACCESS
```

```
ALTER TABLE MEDICAL_RECORD ALTER COLUMN  
MEDICAL_HISTORY SECURED WITH  
MEDICAL_RECORD_POLICY.MED_RECORD ADD COLUMN  
DEPARTMENT_TAG DB2SECURITYLABEL ADD SECURITY  
POLICY MEDICAL_RECORD_POLICY
```

Case Study: MAC Solution

3. Updating the MEDICAL_RECORD table security label column.

```
GRANT EXEMPTION ON RULE DB2LBACWRITETREE FOR  
MEDICAL_RECORD_POLICY TO USER  
<administrator_auth_id>
```

For each department,

```
UPDATE MEDICAL_RECORD set DEPARTMENT_TAG=  
SECLABEL_BY_NAME ('MEDICAL_RECORD_POLICY',  
'DEPT_K01') where DEPTNO='K01'
```

LBAC(Label Based Access Control)

4. Granting the appropriate security labels to users.

```
GRANT SECURITY LABEL MEDICAL_RECORD_POLICY.  
MEDICAL_ANALYST TO USER PETER FOR ALL ACCESS
```

```
GRANT SECURITY LABEL  
MEDICAL_RECORD_POLICY.DEPT_K01 TO USER Andrea FOR  
ALL ACCESS
```

```
GRANT SECURITY LABEL  
MEDICAL_RECORD_POLICY.DEPT_S02 TO USER Joseph for  
ALL ACCESS
```